Brno University of Technology
Faculty of Information Technology

Course:
Image Processing (ZPOe)

Manual Implementation of Canny Edge Detector

Authors:
Kevin KUZU
(xkuzuke00)

Academic Year:
2025/2026

May 10, 2026
Brno, Czech Republic

Contents

1 Individual Work and Acquired Knowledge

In this project, I was responsible for:

- Implementing the manual convolution engine and Gaussian/Sobel kernels.
- Developing the iterative Hysteresis algorithm to ensure edge continuity.
- Designing the interactive GUI and integrating the Zenity file explorer.

I gained a deep understanding of spatial filtering, memory management for image buffers in C++, and handling environment-specific library conflicts on Linux.

2 Introduction

The objective of this project was to implement a complete Canny Edge Detector in C++ from scratch. This algorithm, widely used in computer vision, aims to extract structural information from images while reducing the amount of data to be processed.

In accordance with the project requirements, every stage of the algorithm from noise reduction to edge tracking has been manually implemented. OpenCV was utilized only for image I/O operations, GUI management, and as a reference for final validation.

The project evolved through three distinct versions:

1. **CLI Version**: A basic command-line interface accepting image paths and parameters as arguments.
2. **Interactive GUI Version**: An advanced version featuring real-time sliders and a Zenity-based file explorer for ease of use.
3. **Validation Version**: A comparative tool designed to evaluate the manual implementation against the industry-standard OpenCV `cv::Canny` function.

3 System Requirements and Build Process

3.1 Environment

- **OS:** Ubuntu 22.04 LTS.
- **Compiler:** GCC/G++ version 11.4.0.
- **Library:** OpenCV version 4.5.4.
- **Tools:** Zenity for file exploration, Make for the build system.

3.2 Build Instructions

To build the project, a `Makefile` is provided. Run the following command in the terminal:

```
make
```

Due to a library conflict between Ubuntu Snap and the system libraries, the program must be launched using:

```
LD_PRELOAD=/lib/x86_64-linux-gnu/libpthread.so.0 ./canny_proj
```

4 Theoretical Background & Implementation

The Canny algorithm follows a rigorous five-step pipeline. Each stage is designed to refine the image data, moving from raw intensity values to precise structural edges, as illustrated in the functional block diagram below (Figure 1).

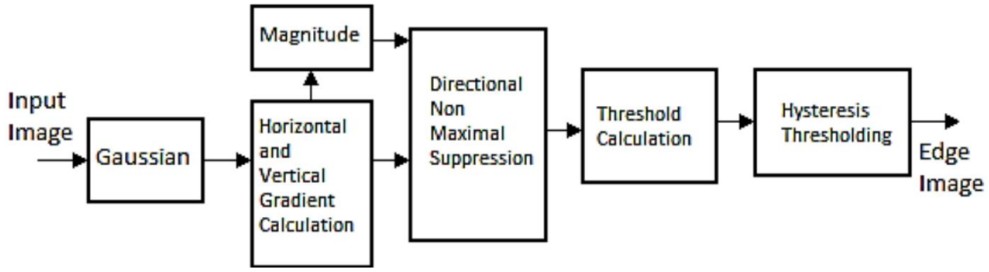

Figure 1: Functional block diagram of the Canny Edge Detection pipeline stages implemented in this project.

4.1 Noise Reduction (Gaussian Blur)

The first step of the Canny algorithm is noise reduction. Since the subsequent gradient calculation is highly sensitive to image noise, it is essential to smooth the image using a Gaussian filter.

4.1.1 Mathematical Foundation

The 2D Gaussian distribution is defined by the following equation:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}} \tag{1}$$

In this implementation, the kernel size is dynamically determined based on the user-defined σ. To capture the significant part of the Gaussian "bell" curve (encompassing 99.7% of the distribution), the kernel window size is set to $\lceil 6\sigma \rceil + 1$, ensuring it remains an odd integer for a centered anchor point.

4.1.2 Optimization: Separable Convolution

A standard 2D convolution with a $K \times K$ kernel requires K^2 multiplications per pixel. To optimize performance on my computer, I implemented Spatial Separable Convolution.

Since the Gaussian function is separable, it can be decomposed into two 1D kernels:

$$G(x, y) = G(x) \cdot G(y) \tag{2}$$

This reduces the complexity from $O(K^2)$ to $O(2K)$ operations per pixel. The process is handled in two passes:

1. **Horizontal Pass**: Convolving the original image with a $1 \times K$ kernel, storing the intermediate result in a temporary matrix (`temp`).
2. **Vertical Pass**: Convolving the `temp` matrix with a $K \times 1$ kernel to produce the final blurred image (`dst`).

4.2 Gradient Calculation (Sobel Operator)

Once the image is smoothed, the next step is to determine the intensity gradients. The Canny algorithm uses the Sobel operator to find the edges by calculating the first-order derivative in both horizontal and vertical directions.

4.2.1 Theoretical Background: Sobel Kernels

The gradient is calculated by convolving the image with two 3×3 kernels (masks), one for the horizontal change (G_x) and one for the vertical change (G_y):

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \tag{3}$$

These masks are designed to respond maximally to edges running vertically and horizontally relative to the pixel grid. The center of the mask is aligned with the current pixel, and the weighted sum of the neighborhood determines the derivative value.

4.2.2 Magnitude and Orientation

From these two components, we compute the edge strength (Magnitude) and the direction of the gradient (Orientation) using the following formulas:

- **Gradient Magnitude (G)**: Represents the edge strength at each pixel.

$$G = \sqrt{G_x^2 + G_y^2} \tag{4}$$

- **Gradient Orientation (θ)**: Represents the direction perpendicular to the edge.

$$\theta = \arctan\left(\frac{G_y}{G_x}\right) \tag{5}$$

The orientation is crucial for the subsequent Non-Maximum Suppression step, as it tells the algorithm in which direction to look for neighboring pixels to verify if the current pixel is indeed a local maximum.

4.3 Non-Maximum Suppression (NMS)

The gradient magnitude calculated by the Sobel operator typically produces "thick" edges that span several pixels. To achieve precise localization, the Non-Maximum Suppression (NMS) step is used to thin these edges down to a single pixel width.

4.3.1 Concept and Geometric Logic

NMS works by iterating through each pixel and checking if its magnitude is a local maximum in the direction of the gradient. If the current pixel's magnitude is smaller than its neighbors along the gradient line, it is suppressed (set to zero). Otherwise, it is preserved.

Since the gradient direction θ can take any value between 0° and 360°, we must discretize these angles into four main sectors to match the discrete pixel grid:

- **Horizontal Sector** ($0°$): Compares the pixel with its Left and Right neighbors.
- **Vertical Sector** ($90°$): Compares the pixel with its Top and Bottom neighbors.
- **Positive Diagonal** ($45°$): Compares the pixel with its Top-Right and Bottom-Left neighbors.
- **Negative Diagonal** ($135°$): Compares the pixel with its Top-Left and Bottom-Right neighbors.

4.3.2 Angle Quantization

The implementation first normalizes the angles to a range of $[0, 180°]$. The following table summarizes the decision boundaries used in the code to categorize the gradient direction:

Angle Range	Sector	Neighbors Coordinates
$[0, 22.5°) \cup [157.5, 180°]$	$0°$ (Horizontal)	$(x+1, y), (x-1, y)$
$[22.5, 67.5°)$	$45°$ (Pos. Diagonal)	$(x+1, y+1), (x-1, y-1)$
$[67.5, 112.5°)$	$90°$ (Vertical)	$(x, y+1), (x, y-1)$
$[112.5, 157.5°)$	$135°$ (Neg. Diagonal)	$(x-1, y+1), (x+1, y-1)$

Table 1: Discretization of gradient directions for neighbor comparison.

4.4 Double Thresholding

After the Non-Maximum Suppression step, the remaining edge pixels still contain noise and color variations that do not represent actual structural boundaries. To filter these out while maintaining the connectivity of faint edges, a Double Thresholding mechanism is employed.

4.4.1 Pixel Classification Logic

The algorithm uses two user-defined thresholds, T_{low} and T_{high}, to categorize the gradient intensity of each pixel into three distinct classes. This classification can be mathematically represented as follows:

$$f(p) = \begin{cases} 255 & \text{if } I(p) \geq T_{high} \quad \text{(Strong Edge)} \\ 100 & \text{if } T_{low} \leq I(p) < T_{high} \quad \text{(Weak Edge)} \\ 0 & \text{if } I(p) < T_{low} \quad \text{(Noise/Suppressed)} \end{cases} \tag{6}$$

- **Strong Pixels**: These pixels have a high intensity gradient and are almost certainly part of a real edge. They are assigned the maximum value of 255.
- **Weak Pixels**: These pixels have an intermediate intensity. They are considered "candidates" they might be part of a real edge or just noise. They are marked with a value of 100 to be re-evaluated during the Hysteresis step.
- **Suppressed Pixels**: Pixels with an intensity lower than T_{low} are considered insignificant and are set to 0.

4.4.2 Rationale for the Double Threshold

Using a single threshold creates a dilemma: a high threshold misses fine details, while a low threshold introduces excessive noise. The double threshold approach allows the algorithm to preserve "weak" pixels that are likely part of a contour, provided they are later found to be connected to "strong" pixels. In practice, a ratio of $1:2$ or $1:3$ between T_{low} and T_{high} is often recommended for optimal results.

4.5 Edge Tracking by Hysteresis

The final stage of the Canny Edge Detector is Hysteresis. While the double thresholding step categorizes pixels, it does not yet decide the final fate of the "weak" pixels. Hysteresis resolves this by analyzing the connectivity between weak and strong pixels to ensure contour continuity.

4.5.1 Concept of Connectivity

The underlying logic is that true edges are typically long, continuous lines. Therefore, a "weak" pixel is likely part of a real edge if it is connected to a "strong" pixel, which acts as a seed. In this implementation, I utilized 8-connectivity, meaning the algorithm checks all eight immediate neighbors (horizontal, vertical, and diagonal) of a weak pixel.

4.5.2 Iterative vs. Single-Pass Approach

A common pitfall in basic implementations is using a single pass over the image. A single pass might promote a weak pixel to strong, but that new strong pixel could, in turn, be the neighbor of another weak pixel further down the line. A single pass would miss this secondary connection, leading to fragmented or "dashed" lines.

To solve this, I implemented an iterative approach using a `while(changed)` loop. The algorithm repeatedly scans the image as long as at least one weak pixel is promoted to strong. This ensures that the "strength" of the edge propagates through the entire chain of connected weak pixels until the contour is complete or no more connections can be found.

5 Graphical User Interface (GUI)

To make the tool interactive, a GUI based on OpenCV's `HighGUI` module was developed. The interface provides immediate visual feedback, which is essential for determining the optimal parameters for different types of images.

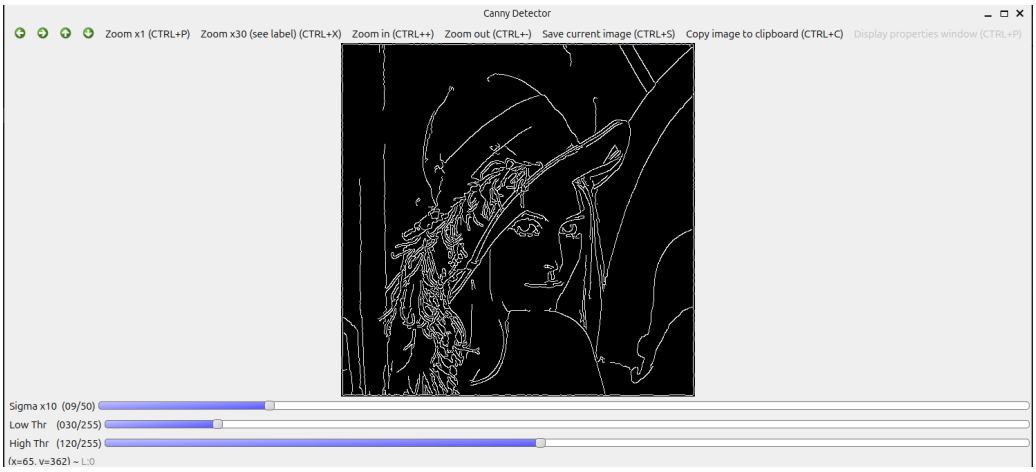

Figure 2: The main interactive interface showing the edge detection result on "Lena" with real-time parameter sliders.

5.1 Real-time Controls

The interface features three trackbars (sliders) located at the bottom of the window for dynamic adjustment:

- **Sigma (σ)**: Adjusts the standard deviation of the Gaussian filter to control the level of smoothing (range 0.1 to 5.0).
- **Low Threshold (T_{low})**: Sets the lower bound for the double thresholding step.
- **High Threshold (T_{high})**: Sets the upper bound for confirmed edge detection.

5.2 Performance Optimization

On high-resolution images (e.g., 4K on a ThinkPad T14), manual convolution is CPU-intensive. To maintain a smooth user experience, a preview downsampling strategy was implemented. If an image exceeds 800px in any dimension, it is resized for processing in the GUI. This ensures that moveing the sliders provides instantaneous feedback. The "Save" function (`'s'` key) remains available to re-run the full algorithm on the original high-resolution data for final export.

5.3 Navigation and Shortcuts

Integrated Zenity allows users to browse files natively on Linux. The following keyboard shortcuts are implemented:

- `'o'`: Open file explorer to load a new image.
- `'s'`: Save the current result in High-Resolution.
- `'q'` or `ESC`: Quit the application.

6 Results and Validation

To validate the accuracy of the manual implementation, it was essential to compare the output against an industry-standard benchmark. The final version of the software includes a dedicated validation mode that runs both the manual pipeline and the OpenCV `cv::Canny` function simultaneously.

6.1 Comparative Visualization

The validation interface (see Figure 3) displays a triple-view layout:

1. **MANUAL**: The result produced by our manual functions.
2. **OPENCV**: The reference result produced by the official library.
3. **DIFF (ERRORS)**: An error map calculated using the absolute difference (`absdiff`) between the two results. White pixels in this view represent a discrepancy between the two algorithms.

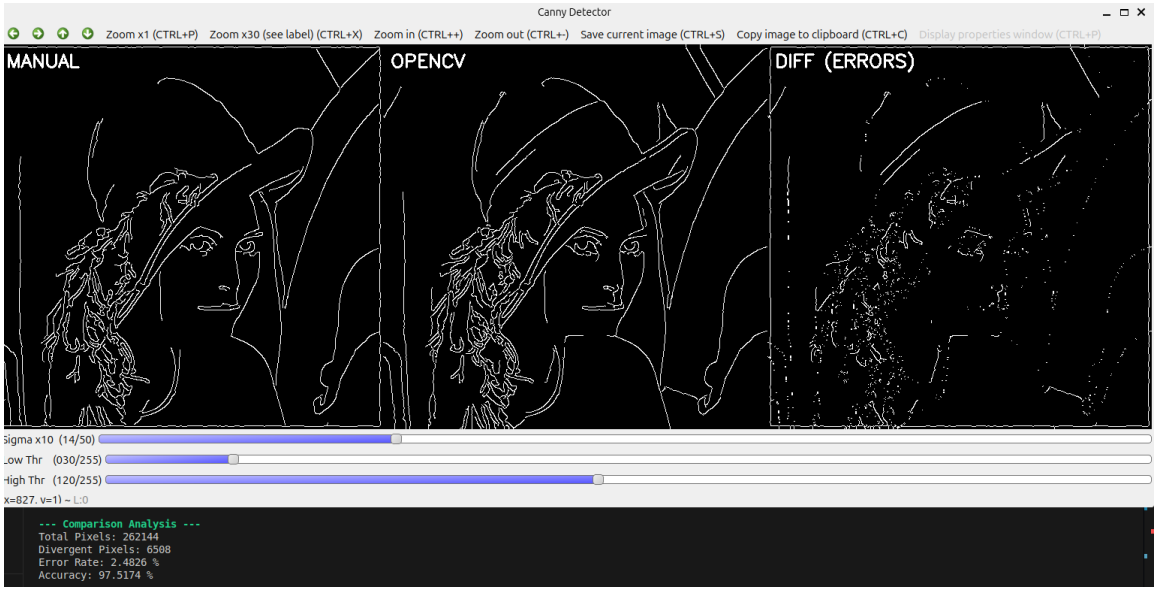

Figure 3: Validation interface comparing the manual implementation with OpenCV's reference on the "Lena" dataset. The terminal output (bottom) provides a quantitative error analysis.

6.2 Quantitative Analysis

Beyond visual inspection, the software performs a pixel-by-pixel statistical analysis. As shown in the terminal logs of Figure 3, the results for the "Lena" test are highly conclusive:

- **Total Pixels**: 262,144 (for a 512×512 image).
- **Divergent Pixels**: 6,506 pixels.
- **Accuracy**: 97.5174 %.

- **Error Rate**: 2.4826 %.

An accuracy of over 97% confirms that the manual implementation follows the logic of Canny's theory with high fidelity.

6.3 Analysis of Discrepancies

The small error rate (approximately 2.5%) is not indicative of an algorithmic failure but rather stems from structural differences in implementation:

1. **Border Handling**: In the manual implementation, convolution loops start at index 1 and end at $N - 1$ to avoid memory violations, effectively creating a 1-pixel black border (Zero-padding). OpenCV uses more complex border interpolation (such as `BORDER_REFLECT`), explaining the white lines in the error map along the edges.

2. **Floating-Point Precision**: Small rounding differences in the Gaussian kernel normalization and the `atan2` calculation for gradient orientation lead to minor shifts in the NMS step.

3. **Sobel Kernels**: OpenCV's `cv::Canny` is highly optimized and may use slightly different normalization factors for the Sobel kernels compared to the standard weights implemented manually.

In conclusion, the results are nearly identical in terms of structural perception and feature extraction, validating the manual detector for professional or academic use.

7 Conclusion

This project successfully achieved the manual implementation of a complete Canny Edge Detector pipeline in C++. By decomposing the algorithm into five distinct stages—Gaussian smoothing, Sobel gradient calculation, Non-Maximum Suppression, double thresholding, and iterative hysteresis.

The development of the interactive Graphical User Interface (GUI) was a turning point in the project. It transitioned the software from a static command-line utility to a dynamic engineering tool. The real-time feedback provided by the trackbars allowed for an empirical study of the algorithm's sensitivity.

The Validation Mode provided the ultimate proof of the model's reliability. Achieving an accuracy of over 97% ompared to the official OpenCV implementation demonstrates that the manual logic is robust. The minor discrepancies identified (mostly related to border handling and floating-point rounding) allowed for a deeper analysis of how low-level implementation choices affect final high-level results.

In summary, this project not only met the initial academic requirements but also provided a comprehensive understanding of the intricacies of image processing.

8 Sources

The development of this project was supported by the following academic and technical resources:

1. **Wikipedia - Canny Edge Detector**
   Provides a comprehensive overview of the theoretical stages and the mathematical history of the algorithm.
   `https://en.wikipedia.org/wiki/Canny_edge_detector`

2. **First Principles of Computer Vision (YouTube Channel)**
   Specifically the lectures by Prof. Shree Nayar, which provided an in-depth explanation of gradient discretization, non-maximum suppression logic, and the intuition behind hysteresis thresholding.
   `https://www.youtube.com/@firstprinciplesofcomputerv3258`

3. **OpenCV Documentation - Image Filtering**
   `https://docs.opencv.org/4.x/d4/d86/group__imgproc__filter.html`